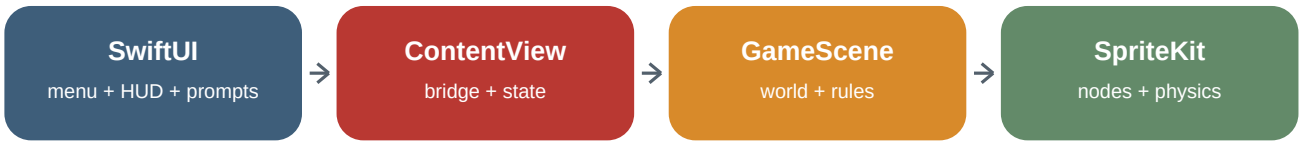
Deliver While Hot

A plain-language guide to the project's SwiftUI, SpriteKit, game rules, tests, and diagnostics



State flows down; user actions and scene events flow back up.

What this project is

Deliver While Hot is a portrait arcade prototype. SwiftUI owns the menu and HUD. SpriteKit owns the scrolling road, characters, collisions, and effects. The project is intentionally small enough to learn from: most behavior can be followed from the scene update loop into a node or a pure rule function.

The useful mental model

The screen is a view of the game state. Input changes intent, GameScene advances the world, GameRules calculates numbers, and the UI redraws from the resulting state. Keeping those responsibilities separate makes bugs easier to locate.

Start here

- **StartView.swift** - menu, How to Play, and About.
- **ContentView.swift** - connects the SpriteKit scene to SwiftUI.
- **GameScene.swift** - the main coordinator and frame-by-frame update loop.
- **GameRules.swift** - small, testable calculations such as food, health, and temperature.
- **HazardNode.swift** and related nodes - visuals, movement, and collision bodies.

How one frame works

Every frame repeats this loop while the game is active.

1. Update

SpriteKit calls **GameScene.update(_:)** once per frame. The scene measures elapsed time and limits very large time jumps so a pause does not teleport everything.

2. Move

The scene moves the world downward to create the road-scroll illusion. The rider's input becomes a target velocity, and smoothing eases toward it so steering is less abrupt.

3. Collide

SpriteKit physics reports contacts. The scene decides what the contact means: damage, block feedback, a food change, a projectile reflection, or an explicit removal.

4. Publish

The scene sends a new **GameState** to SwiftUI. The HUD then displays hearts, food slices, temperature, cooldowns, and ending prompts.

SpriteKit words in plain English

An **SKNode** is a thing in the scene. A sprite node draws an image; a parent node groups children; a physics body gives a node a collision shape; z-position decides what is drawn in front. An **SKAction** changes a node over time, such as moving or fading it. For animation atlases, the node stays the same size while only its texture changes.

Layer order

The world is drawn behind gameplay. Road and sidewalks sit behind lane markings. Walls and props sit above the ground but below hazards. The rider and hazards sit above scenery. Rain sits over the world but below HUD and controls. SwiftUI draws the HUD and buttons above the SpriteKit view.

Rules, tests, and safe changes

Pure rules are the easiest code to test because they do not need a simulator. For example, GameRules can be given a state with one quarter of food and asked to apply a blocked hit. The test checks the number without needing to draw a road or tap a button.

Current safety net

- **GameRulesTests.swift** covers temperature floor, health floor, bribes, blocked hits, pigeon loss, pizza recovery, eating, and delivery outcome.
- **MenuAndGameplayUITests.swift** opens menu information, starts a run, and checks that gameplay controls appear.
- The GitHub Actions workflow runs the Xcode test scheme on an available iPhone simulator.

What still needs manual testing

- Sprite alignment, crop boundaries, animation size, and visual seams.
- Steering feel and hazard spacing at real device scale.
- The Prepare then Jump ending sequence, including success and failure timing.
- Audio behavior when the app backgrounds or the run ends.

A practical debugging routine

First reproduce the issue with one short run. Then log the state transition, the node position, and the phase name. Change one variable at a time. Rebuild, repeat the same input, and compare the result. If a calculation can be moved into GameRules, do that: it makes the next test cheap.

Repository hygiene

Source files belong in Git. DerivedData, .build output, xcuserdata, and user interface state do not. The repository now ignores those generated files. The app also uses a neutral bundle identifier and does not contain a personal development Team ID.

Optional diagnostics

DiagnosticsReporter is intentionally small. It logs an event to Apple's local OS log and only sends a JSON event when the developer supplies an endpoint through the launch environment. There is no bundled DSN, API key, account, or user identifier.

Event flow

Game starts -> reporter records *app_launched* -> local OS log is written -> if an endpoint exists, a background POST sends event name, app version, OS version, and explicit metadata -> failures are logged without interrupting gameplay.

Try it locally

In Xcode, edit the DeliverWhileHot scheme, open Run > Arguments > Environment Variables, and add `DELIVER_WHILE_HOT_MONITORING_ENDPOINT` with a test collector URL. Leave it absent for normal runs. A production provider such as Sentry or Firebase Crashlytics should be selected only after privacy, retention, and redistribution decisions are documented.

Public sharing checklist

- Run the build and test scheme on a clean checkout.
- Record a short simulator video and note known prototype limitations.
- Verify the exact license and attribution terms for every audio, image, and font asset.
- Do not treat a public download page as proof that redistribution is allowed.
- Keep monitoring disabled until a real endpoint and privacy policy exist.

The right next milestone

For a portfolio presentation, stop when a reviewer can clone the repository, understand the architecture from the README, build the app, run the tests, and play one complete run. More polish can come after that baseline is reproducible.

Generated for the Deliver While Hot prototype on October 10, 2026.